

Adaptive Hybrid Retrieval-Augmented Generation

Optimizing the computational cost-performance trade-off via learned query routing

Dante Wesslund · 2026-10-22

Examiner: Pawel Herman · Supervisors: Arvid Eriksson (KTH), Peng Zhang (Ericsson)

DA233X Degree Project, Machine Learning · In collaboration with Ericsson

In short

Route before you retrieve. The question alone often tells you whether the expensive retrieval is needed.

42%
less time than always using Mix

63%
fewer tokens

92%
of correct answers kept

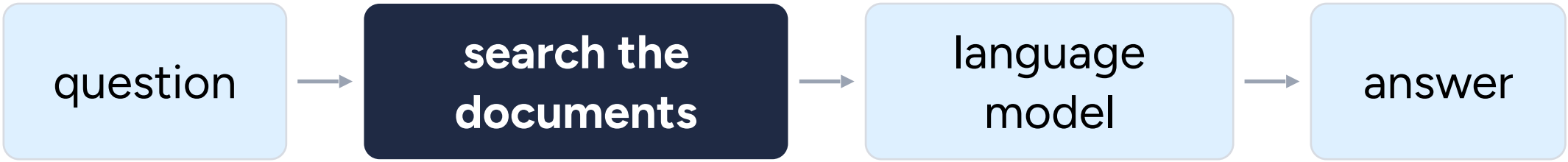
1 The problem

2 The method

3 The findings

4 What it means

Retrieval-augmented generation

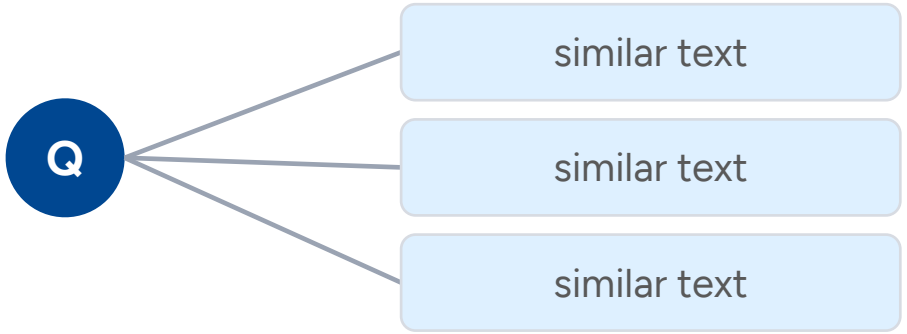


The answer is only as good as **what the search finds**.

Two ways to search

Naive

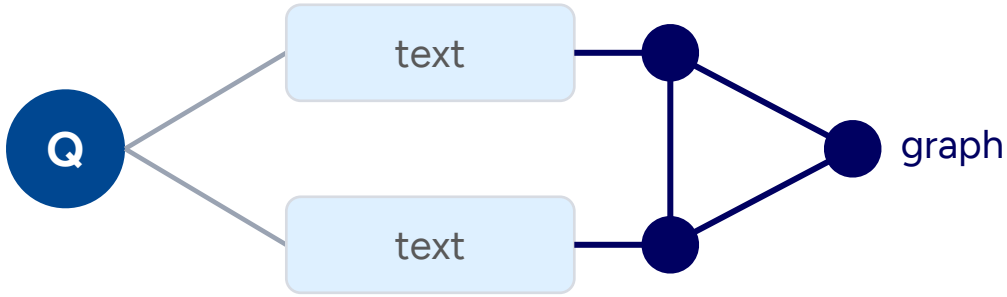
vector search only



10.9 s per question · **≈3,500** tokens

Mix

vector search + knowledge graph



24.4 s per question · **≈17,900** tokens

Mix can connect facts across documents, at **2.2× the time** and **5× the tokens**.

Some questions need Mix

Of 2,000 judged questions, Mix was needed on only **327**.

Which film came out first, The Love Route or Engal Aasan?	Naive correct	Mix correct	Naive is enough
When was the director of the film Private Hell 36 born?	Naive not enough	Mix correct	**needs Mix**

Naive found the director, Don Siegel, but not his birth date. Mix answered: October 26, 1912.

Query routing



Decide from the **question alone**, before paying for retrieval.

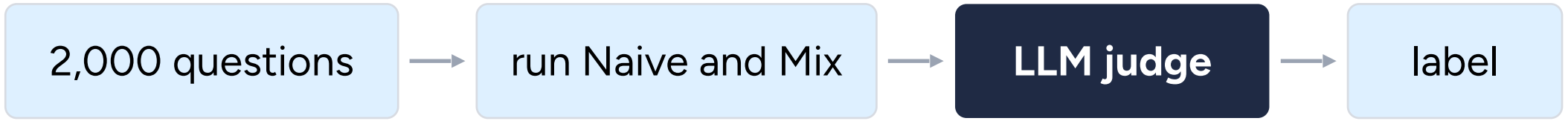
Related: Adaptive-RAG (Jeong et al., 2024), RouterBench (Hu et al., 2024)

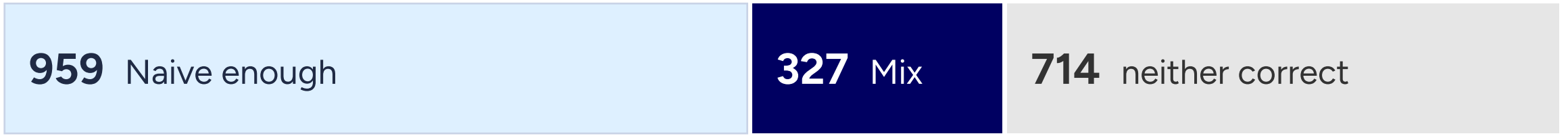
Research question

Within one Hybrid RAG system, how well can a **lightweight classifier** predict from the question alone whether the cheap retrieval mode is enough, and how much **cost** does routing with it save?

1. **Prediction.** ModernBERT vs a TF-IDF baseline and a question-type rule
2. **Cost.** Time and tokens saved vs always using Mix

Building the labels





Label = the **cheapest mode that was enough**. The 1,286 questions where at least one mode worked are used.

The setup

Data

1,286 labelled questions from 2WikiMultihopQA, split 900 / 193 / 193 for training, validation and test.

Retrieval

LightRAG: Naive and Mix on one index. Gemini writes and judges the answers.

Routers

TF-IDF with logistic regression as the baseline, and a fine-tuned ModernBERT: three training runs, scores averaged.

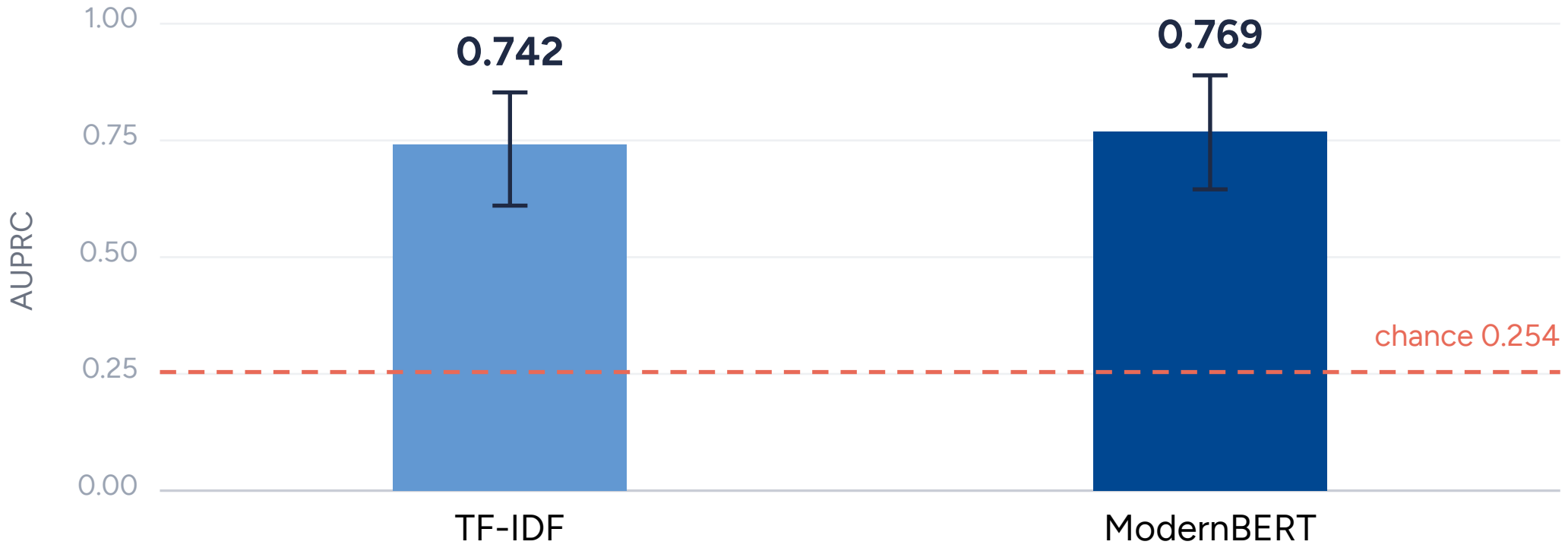
Evaluation

On the 193 test questions: prediction quality and routing cost. Paired bootstrap and permutation tests, Holm-corrected.

Can the question predict it?

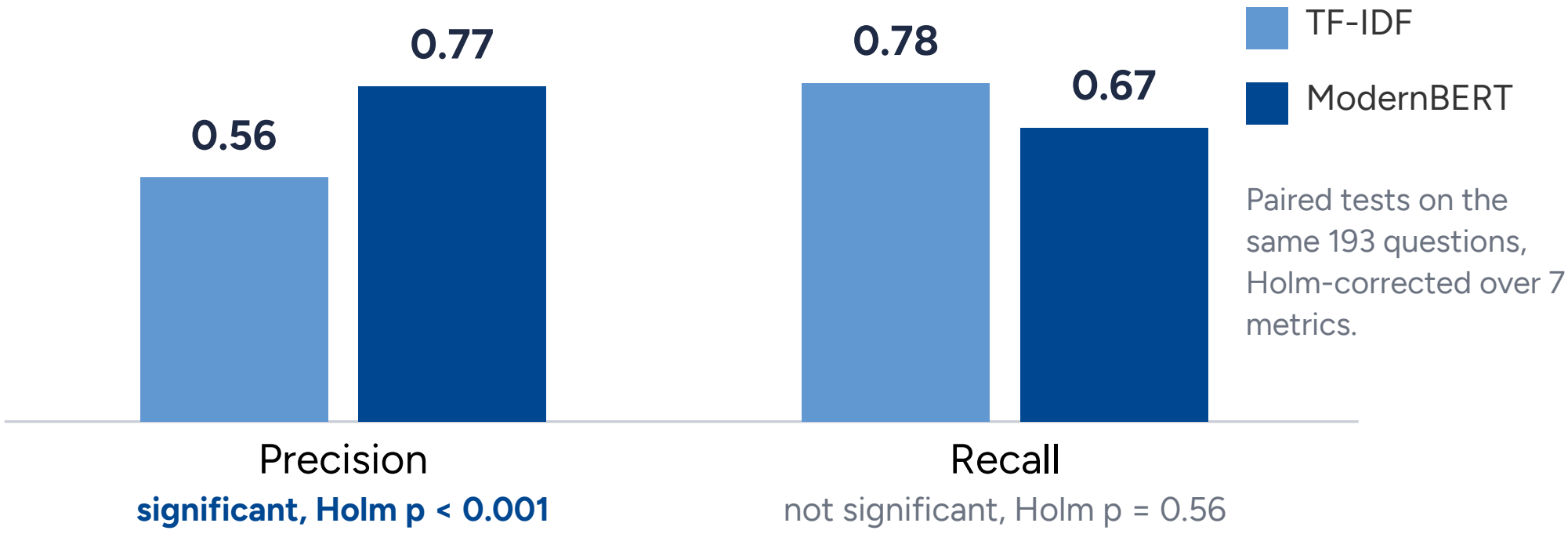
Yes, partly. Both routers find the Mix-needing questions **far above chance**.

Test set: 193 questions, 49 need Mix. Whiskers: 95% bootstrap intervals.



More selective

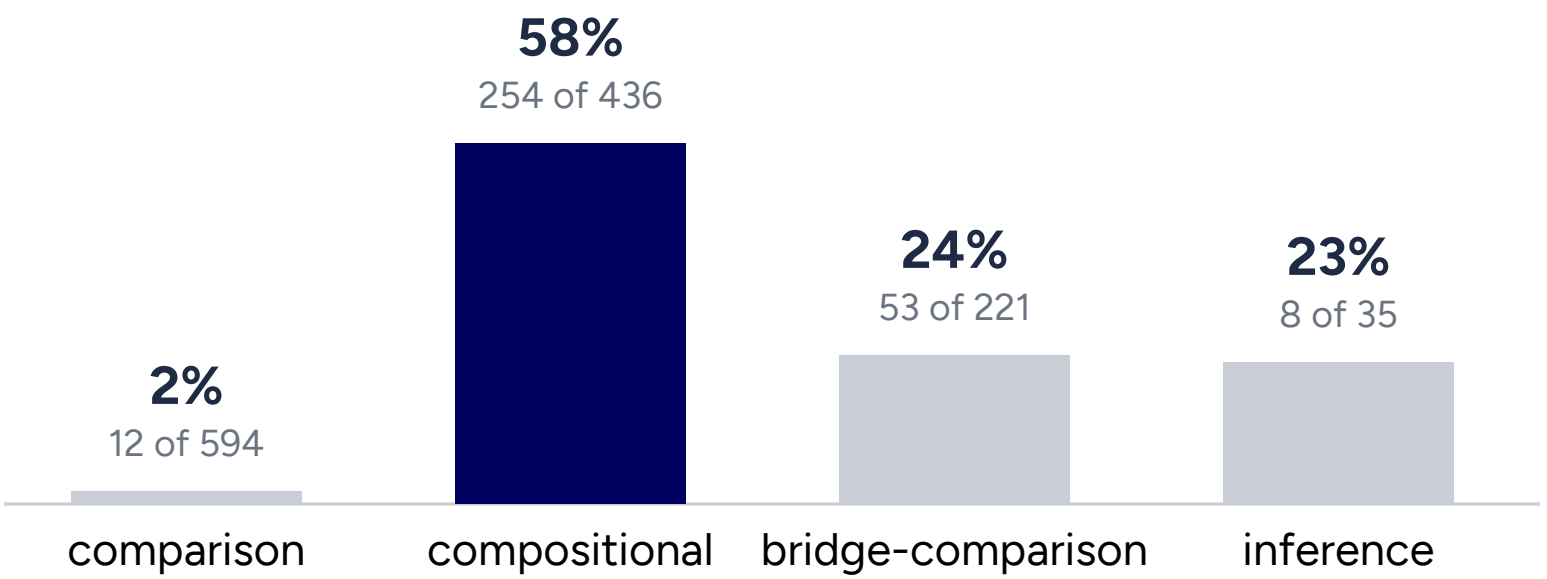
ModernBERT is **significantly more precise and more accurate**. Recall, F1 and the ranking metrics are not resolved.



Where the signal comes from

Largely **question structure**, recognised from the wording.

Share of the 1,286 labelled questions that needed Mix, by type



Precision (test set)

Question-type rule **0.58**

TF-IDF **0.56**

ModernBERT **0.77**

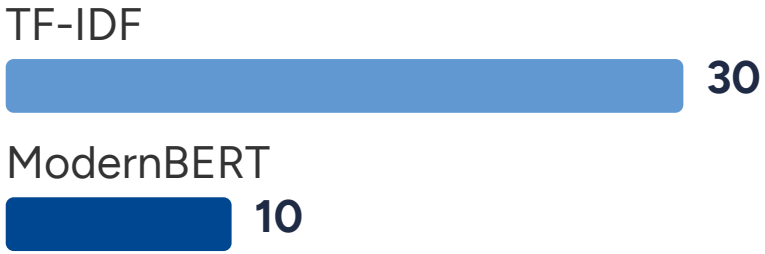
The rule needs a type label that real questions lack. Only ModernBERT is significantly above it.

Routing errors

Far less wasted compute, for a small and unresolved increase in risk.

Unnecessary Mix calls

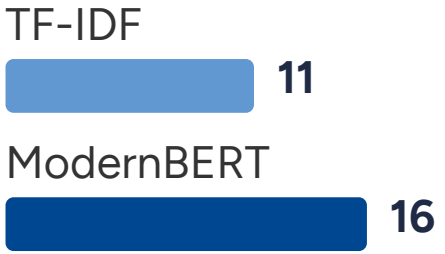
of 144 questions where Naive was enough



significant, McNemar p < 0.001

Missed Mix calls

of 49 questions that needed Mix



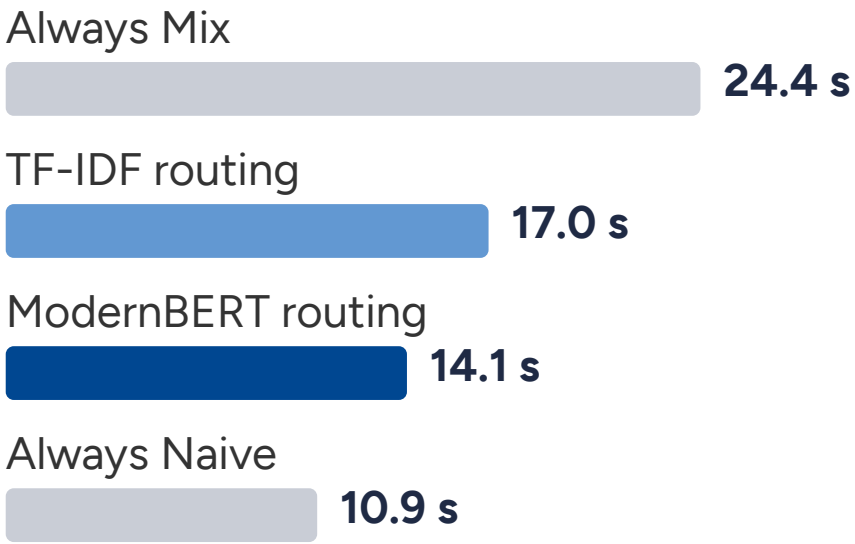
not significant, McNemar p = 0.125

Cost

ModernBERT routing: **42% less time** and **63% fewer tokens** than always-Mix.

Mean time per question

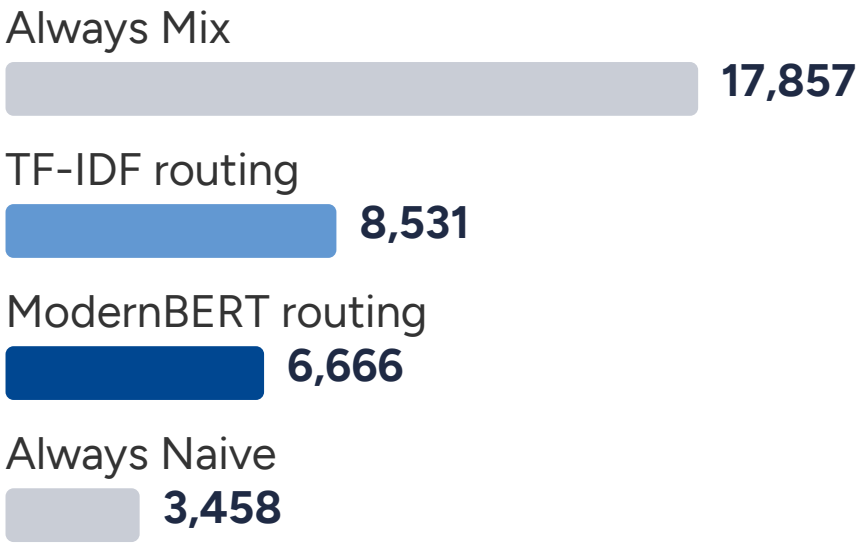
measured on every test question



ModernBERT vs TF-IDF: 2.9 s faster, p = 0.003

Model tokens per question

estimated from 15 instrumented questions

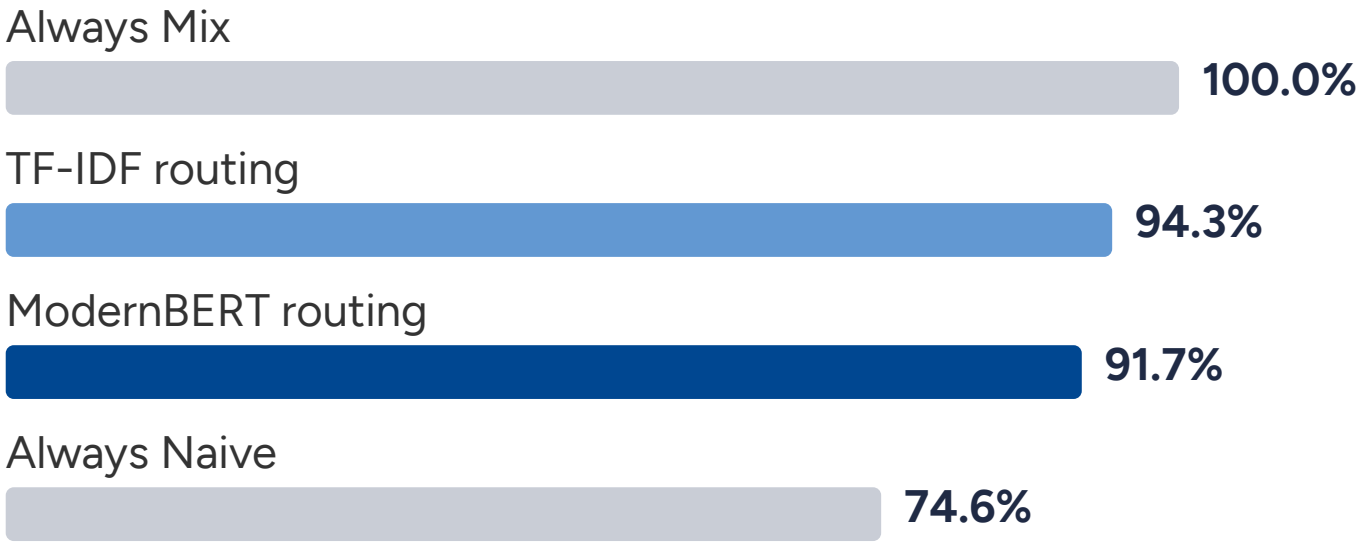


Answers kept

About **92% of the correct answers**, at 37% of the token cost.

Share of always-Mix's correct answers kept

assuming Mix is correct wherever Naive was



In practice

Prior work

Extends Adaptive-RAG to vector vs graph retrieval. As in RouterBench: most of the quality, a fraction of the cost.

Deployment

No language-model call per question. The set-up cost is the labels. Needs a small labelled set from the target workload first.

A cost dial

The threshold trades wasted Mix calls against missed ones. Each deployment sets its own.

Limitations

- **Labels.** An LLM judge decided which mode was enough, using the same Gemini model that wrote the answers, without a human check.
- **Estimates.** Tokens come from 15 instrumented questions, answers kept assume Mix is right wherever Naive was, and router time was not measured.
- **Scope.** One benchmark and one framework, a random split that can share question templates, and only 193 test questions.

Future work

- **Check the labels.** Validate a sample of judge decisions by hand, and judge again with a model from another family.
- **Measure everything.** Log tokens for every question, time the router itself, and re-judge the answers each routing policy picks.
- **Go further.** Other benchmarks and frameworks, a larger test set, and a second stage that escalates to Mix when Naive's retrieval looks weak.

Conclusions

Route before you retrieve. The question alone often tells you whether the expensive retrieval is needed.

- **Prediction.** Both routers find Mix-needing questions far above chance. At its chosen threshold, ModernBERT is significantly more precise than TF-IDF and the type rule.
- **Cost.** 42% less time and an estimated 63% fewer tokens than always using Mix, keeping about 92% of correct answers, under an assumption.
- **Scope.** One benchmark and judged labels. Validate on real queries before deploying.



Thank you

Questions?

github.com/Dantecool02/Thesis_ericsson